Darshan UNIVERSITY

योग: कर्मसु कौशलम्

Python Programming - 2301CS404

Lab - 12

Roll No : 496

Name : Dhruv Limbasiya

Enrollment : 24031701027

Exception Handling

01) WAP to handle following exceptions:

1. ZeroDivisionError
2. ValueError
3. TypeError

Note: handle them using separate except blocks and also using single except block too.

In [14]:
```
try:
    num1 = int(input("Enter first number: "))
    num2 = int(input("Enter second number: "))

    result = num1 / num2
    print("Result:", result)

except ZeroDivisionError as e:
    print("Error: Cannot divide by zero." , e)

except ValueError as e:
    print("Error: Invalid input! Please enter integers only." , e)

try :
    num1 = int(input("Enter first number: "))
    num2 = 'aa'
    result = num1 / num2
    print("Result:", result)

except TypeError as e:
    print("Error: Type mismatch occurred." , e)
```

```
Error: Cannot divide by zero. division by zero
Error: Type mismatch occurred. unsupported operand type(s) for /: 'int' and 'str'
```

02) WAP to handle following exceptions:

1. IndexError
2. KeyError

In [15]:
```
try:
    list1 = [10, 20, 30]
    index = int(input("Enter index number: "))
    print("Value at index:", list1[index])

except IndexError as err:
    print("Error: Index out of range." , err)
```

```
try :
    dict1 = {"name": "Dhruv", "age": 20}
    key = input("Enter key to search in dictionary: ")
    print("Value for key:", dict1[key])

except KeyError as err:
    print("Error: Key not found in dictionary." , err)
```

```
Error: Index out of range. list index out of range
Error: Key not found in dictionary. 'wer'
```

03) WAP to handle following exceptions:

1. FileNotFoundError
2. ModuleNotFoundError

In [16]:
```
try:
    file = open("file1.txt", "r")
    data = file.read()
    print("File Content:\n", data)
    file.close()

except (FileNotFoundError, ModuleNotFoundError) as e:
    print("Error occurred:", e)
```

```
Error occurred: [Errno 2] No such file or directory: 'file1.txt'
```

04) WAP that handles all type of exceptions in a single except block in standard error message format.

In [25]:
```
try:
    result = 10 / 0
    print("Result:", result)

    l1 = [10, 20, 30]
    print("Value at index:", l1[6])

    d1 = {"name": "Dhruv", "age": 20}
```

```
    print("Value:", d1["phone"])

    file = open("errorfile.txt", "r")
    content = file.read()
    print("File Content:\n", content)
    file.close()

    import abc

except Exception as err:
    print(type(err).__name__, " : " , err)
    print(err)
    print(err.args)
    print(err.args[0])
    print(err.__str__())
    print(type(err))
```

```
ZeroDivisionError  :  division by zero
division by zero
('division by zero',)
division by zero
division by zero
<class 'ZeroDivisionError'>
```

05) WAP to demonstrate else and finally block.

In [18]:

```
try:
    num1 = int(input("Enter first number: "))
    num2 = int(input("Enter second number: "))

    result = num1 / num2

except ZeroDivisionError as err:
    print("Error:" , err)

except ValueError:
    print("Error: " , err)

else:
    print("Division successful.")
```

```
    print("Result:", result)

finally:
    print("This block always executes.")
```

```
Error: division by zero
This block always executes.
```

06) WAP to create an udf divide(a,b) that handles ZeroDivisionError.

In [22]:
```
def divide(a,b):
    try:
        result = a / b
    except ZeroDivisionError as err:
        print("Error :" , err)
    else :
        return result

try:
    num1 = int(input("Enter first number: "))
    num2 = int(input("Enter second number: "))
except Exception as err:
    print("Error :" , err)
else :
    print("Division : ", divide(num1 , num2))
finally :
    print("Program Completed")
```

```
Error : division by zero
Division :  None
Program Completed
```

07) WAP to accept item prices and calculate total. Raise ValueError if price is negative, otherwise print the total bill amount.

In [30]:
```
try :
    total_items = int(input("Enter Total Items :"))
    prices = []
    for i in range (total_items):
```

```
        temp = int(input("Enter price of Item :"))
        if temp < 0 :
            raise ValueError("Price can't be Negative")
        else :
            prices.append(temp)
except ValueError as err :
    print("Error :" , err)
except Exception as err :
    print("Error :" , err) ## for any other errors if they occur
else :
    print("total bill : " , sum(prices))
```

```
total bill :  60
```

08) Create a short program that prompts the user for a list of grades separated by commas.

Split the string into individual grades and use a list comprehension to convert each string to an integer.

You should use a try statement to inform the user when the values they entered cannot be converted.

In [36]:
```
try:
    marks = list(map(int , input("Enter grades separated by commas").split(",")))

except ValueError as err :
    print("Error : values entered cannot be converted to integer")
except Exception as err:
    print("Error :" , err)
else :
    print(marks)
finally:
    print("Program Completed")
```

```
Error : values entered cannot be converted to integer
Program Completed
```

09) Accept 5 subject marks (0-100). Use assert for validation.

Sample Input (Error Case)

Enter marks: 110

Sample Output (Error Case)

AssertionError: Marks should be between 0 and 100.

Sample Input (Normal Case)

Valid marks entered

Sample Output (Normal Case)

Average Marks displayed

In [6]:
```
try :
    marks = []
    for i in range(5):
        temp = int(input("Enter mark of Subject :"))
        assert 0<=temp and temp<=100, "Marks should be between 0 and 100."
        marks.append(temp)
except AssertionError as err :
    print("Assertion Error :" ,err)
except Exception as err:
    print("Error :" ,err)
else:
    print("Average Marks :" , sum(marks)/5)
```

```
Assertion Error : Marks should be between 0 and 100.
```

10) WAP that gets password from user. Password must have at least 8 characters and one digit.

Raise WeakPasswordError if invalid, print the messsage "Correct" otherwise.

In [11]:
```
class WeakPasswordError(Exception):
    pass

try :
    password = input("Enter Password :")
    if len(password)<8:
        raise WeakPasswordError("Password must have at least 8 characters.")
```

```
    if not any(i.isdigit() for i in password):
        raise WeakPasswordError("Password must have at least 1 Digit.")
except WeakPasswordError as err:
    print("WeakPasswordError :" ,err)
except Exception as err:
    print("Error : " ,err)
else :
    print("Correct.")
```

```
WeakPasswordError : Password must have at least 1 Digit.
```

11) WAP to raise your custom Exception named NegativeNumberError with the error message : "Cannot calculate the square root of a negative number" :

if the given number is negative.

otherwise print the square root of the given number.

In [13]:

```
import math
class NegativeNumberError(Exception):
    pass
try:
    num=int(input("Enter number for Square Root:"))
    if num<0:
        raise NegativeNumberError("Cannot calculate the square root of a negative number")
except NegativeNumberError as err:
    print("NegativeNumberError :" , err)
except Exception as err:
    print("Error : " ,err)
else:
    print(math.sqrt(num))
```

```
NegativeNumberError : Cannot calculate the square root of a negative number
```

In [ ]: